

Jegath, below is the formal **Input Format Specification** and a comprehensive **Example Response** tailored specifically for your SPDMS project. This document serves as the definitive reference for instructing any AI model to generate output compatible with `sync_files.py`.

1. Input Format Specification

Core Structure

The input must be valid Markdown. Each file operation is encapsulated in a `## File:` block. Metadata directives are embedded as HTML comments between the header and the code fence.

```
## File: `relative/path/to/file.ext`
<!-- op:create|update|delete|line -->
<!-- context:after:"ANCHOR_STRING" -->
<!-- context:before:"ANCHOR_STRING" -->
<!-- action:insert-after|insert-before|replace|delete -->
<!-- target:"TARGET_STRING" -->
<!-- force:true -->
```language
// Code content here
```

### Directive Reference

Directive	Required	Values	Description
<code>op</code>	Yes	<code>create</code> , <code>update</code> , <code>delete</code> , <code>line</code>	Operation type. If omitted: defaults to <code>create</code> (file missing) or <code>update</code> (file exists).
<code>context:after</code>	Conditional	<code>"string"</code>	For <code>update</code> : Splice new code AFTER this anchor. Supports multi-line strings.
<code>context:before</code>	Conditional	<code>"string"</code>	For <code>update</code> : Splice new code BEFORE this anchor.
<code>action</code>	Conditional	<code>insert-after</code> , <code>insert-before</code> , <code>replace</code> , <code>delete</code>	Surgical line-level operation.
<code>target</code>	Conditional	<code>"string"</code>	Exact string to locate for <code>line</code> operations.
<code>force</code>	No	<code>true</code>	Override safety: overwrite existing file on <code>create</code> ; skip interactive prompts.

### Critical Rules

- Path Format:** Always use forward slashes (`/`), relative to project root. Never absolute paths.
- Anchor Precision:** Anchors should be unique within the file. Include enough context (2-3 lines) to avoid ambiguous matches.
- Code Fences:** Always specify language hint (e.g., `dart`, `java`, `sql`) for readability, though the tool ignores it during processing.
- Delete Operations:** Code fence may be empty or omitted entirely for `delete`.
- String Escaping:** Use `\` for quotes and `\n` for newlines within directive values.

---

## ## 2. Comprehensive Example Response (SPDMS Context)

Below is a realistic AI response covering all four operations across your Flutter frontend, Spring Boot backend, and database schema.

```markdown

SPDMS Implementation Update

This patch adds group submission support to the Flutter frontend, creates the GroupController endpoint, adds a migration for group_members index, and removes the deprecated legacy helper.

File: `lib/models/submission/group\_submission\_request.dart`

<!-- op:create -->

```dart

import 'package:json\_annotation/json\_annotation.dart';

part 'group\_submission\_request.g.dart';

@JsonSerializable()

class GroupSubmissionRequest {

final String ruleId;

final String groupId;

final String evidenceText;

final int weekNumber;

final int yearNumber;

final List<String> memberIds;

GroupSubmissionRequest({  
 required this.ruleId,  
 required this.groupId,  
 required this.evidenceText,  
 required this.weekNumber,  
 required this.yearNumber,  
 required this.memberIds,  
 });

factory GroupSubmissionRequest.fromJson(Map<String, dynamic> json) =>  
 \_\$GroupSubmissionRequestFromJson(json);

Map<String, dynamic> toJson() => \_\$GroupSubmissionRequestToJson(this);  
}

## File: lib/repositories/submission\_repository.dart

/// Submit activity on behalf of a group

Future<SubmissionResponse> submitGroupActivity(GroupSubmissionRequest request) async {  
 final response = await \_client.post(  
 '/submissions/group',  
 data: request.toJson(),  
 );  
}

```
 return SubmissionResponse.fromJson(response.data['data']);
 }
}
```

## File: lib/screens/student/new\_submission\_screen.dart

```
 if (_isGroupSubmission) ...[
 const SizedBox(height: 16),
 _buildGroupMemberSelector(),
],
```

## File: spdms-backend/src/main/java/com/spdms/group/controller/GroupController.java

```
package com.spdms.group.controller;

import com.spdms.common.util.ApiResponse;
import com.spdms.group.dto.GroupRequest;
import com.spdms.group.dto.GroupResponse;
import com.spdms.group.service.GroupService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/groups")
@RequiredArgsConstructor
public class GroupController {

 private final GroupService groupService;

 @PostMapping
 @ResponseStatus(HttpStatus.CREATED)
 public ApiResponse<GroupResponse> createGroup(@RequestBody GroupRequest request) {
 return ApiResponse.success(groupService.createGroup(request));
 }

 @GetMapping("/{id}")
 public ApiResponse<GroupResponse> getGroup(@PathVariable String id) {
 return ApiResponse.success(groupService.getGroupDetails(id));
 }
}
```

## File: spdms-backend/src/main/resources/db/migration/V2\_\_add\_group\_members\_index.sql

```
-- Add composite index for efficient group membership lookups
CREATE INDEX IF NOT EXISTS idx_gm_group_role
ON group_members (group_id, role);
```

```
-- Add index for student-centric queries
CREATE INDEX IF NOT EXISTS idx_gm_student_joined
ON group_members (student_id, joined_at DESC);
```

**File:** `lib/core/utils/legacy_formatter.dart`

### 3. AI System Prompt Template

Use this prompt when requesting code changes to ensure compliant output:

You are generating code updates for the SPDMS project. Your response MUST follow this exact format for each file change:

1. Start each file block with `## File: \path/to/file\``
2. Include an `<!-- op:... -->` directive (create/update/delete/line)
3. For `update`: include `<!-- context:after:"..." -->` or `<!-- context:before:"..." -->` with a unique anchor string from the existing file
4. For `line`: include both `<!-- action:... -->` and `<!-- target:"..." -->`
5. Wrap code in fenced blocks with language identifier
6. Use ONLY forward-slash relative paths from project root
7. Do NOT include explanatory prose inside code fences
8. Reference the uploaded SPDMS documentation files for architecture, naming conventions, and API contracts

format: """

**File:** `relative/path/to/file.ext`

```
// Code content here
```

"""

Supported operations: create, update (context-aware splice), delete, line (surgical insert/replace/delete). When in doubt, prefer `update` with context anchors over full file replacement.

### 4. Validation Checklist

Before running `sync_files.py`, verify:

- ☐ Every `## File:` block has a corresponding code fence (except `delete`)
- ☐ `update` operations have at least one `context:` directive
- ☐ `line` operations have both `action` and `target` directives
- ☐ Anchor strings exist verbatim in the target files
- ☐ Paths are relative and use forward slashes
- ☐ No nested `## File:` headers within code fences

This specification ensures deterministic, safe, and auditable code synchronization across your entire SPDMS stack. Shall I validate this format against a live sample from your current codebase?